HLS implementation of an autoencoder scheme

ELEC-Y541 - Reconfigurable architectures

Emmeran Colot

2026

1 Introduction

The objective of this report is to showcase the work done in an effort to implement an autoencoder (AE) architecture on an FPGA using high-level synthesis (HLS) tools. The model uses the MNIST dataset, consisting of 28x28 black and white pixels of handwritten numbers, and attempts to compress those images while minimizing the difference between the original image and the reconstructed one.

A trained model, written with PyTorch, was already given and only inference had to be run on the final implementation. The model first had to be translated in C++, which is explained in Sec. 2. The resulting implementation then has been tuned with different synthesis directives, optimizing either the performances in Sec. 3 or the area in Sec. 4.

2 C++ Implementation

2.1 Auto-encoder Model Structure

The model consists of 4 encryption layers and 4 decryption layers. Each layer consists of a linear part where the input is multiplied by a *weights* matrix, and then added to a *bias* vector. This linear operation is then followed by an activation function, which is a rectified linear unit (ReLU) for all layers except for the output layer.

Each layer can therefore be implemented by calling `Layer` (given below) with the right sizes and weights/biasses. It can be mathematically summarized as

$$y_i = \begin{cases} 0 & \text{if} \quad (\mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i) < 0 \\ \mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i & \text{otherwise,} \end{cases} \tag{1}$$

where $\mathbf{u}$ and $\mathbf{y}$ are respectively the input and output vector, $\mathbf{W}$ the weights matrix and $\mathbf{b}$ the bias vector.

2.2 Layer Specialization

A first issue that arose from the basic approach of a single function called for every layer is the dynamic number of iterations in the **L0** and **L1** loops, preventing loop pipelining as it would

```
void Layer(float layerInput[IN_SIZE],
            float layerOutput[OUT_SIZE],
            float weights[OUT_SIZE][IN_SIZE],
            float bias[OUT_SIZE]) {
    L0:for(int row = 0; row < OUT_SIZE; row++) {
        L1:for(int col = 0; col < IN_SIZE; col++) {
            layerOutput[row] = layerOutput[row] + weights[row][col] *
               layerInput[col];
        }
        layerOutput[row] = max(0.0f, layerOutput[row] + bias[row]); //
           ReLU
    }
}
```

need to have an undetermined number of stages. For this reason, each type of layer[1] has a specific function with fixed sizes.

2.3 Stream Model

For area optimization, FIFOs should be used whenever possible. To allow to have only FIFOs in a layer, each input and output must be read from / written to a single time and in a sequential order. For this reason, `Layer` has been written with a buffer for the input as it was the only variable being used not sequentially.

```
void Layer(float layerInput[IN_SIZE],
            float layerOutput[OUT_SIZE],
            float weights[OUT_SIZE][IN_SIZE],
            float bias[OUT_SIZE]) {
    float in_buffer[IN_SIZE];
    float tmp;
    L0:for(int row = 0; row < OUT_SIZE; row++) {
        tmp = bias[row];  // Start with bias
        L1:for(int col = 0; col < IN_SIZE; col++) {
            if (row == 0) {
                // Save to buffer if first encounter
                in_buffer[col] = layerInput[col];
            }
            tmp = tmp + weights[row][col] * in_buffer[col];
        }
        layerOutput[row] = max(0.0f, tmp); // ReLU
    }
}
```

2.4 Autoencoder Code

Apart from the layers, the full code contains a first flattening operation in the function *flatten*, which converts the loaded image from a 28x28 matrix into a single vector of 784 elements and that removes the input mean and variance.

[1] a *type of layer* designates a layer with a given input and output layer size.

The opposite operation, applied by the function *unflatten* is applied at the output, reshaping it in a matrix.

3 Layer-wise Optimization

Before optimizing the whole model, a single layer was optimized using HLS directives. This gave a more precise understanding of the impact of the used directives.

3.1 Baseline

The baseline was taken by disabling pipelining and loop flattening on both **L0** and **L1** loops. It used the code introduced in Sec. 2.2.

3.2 Performance Oriented

3.2.1 Inner Loop Pipelining

By pipelining the loop **L1**, each iteration tries to be completed in a single cycle (II = 1 by default) to process the next iteration right afterwards.
There is however a feedback in the loop as the output of the layer is updated for each row of the input. The math being run in floating point, the addition takes multiple cycles and the resulting design can only assure an initiation interval (II) of 3 cycles, as shown on Fig. 1.

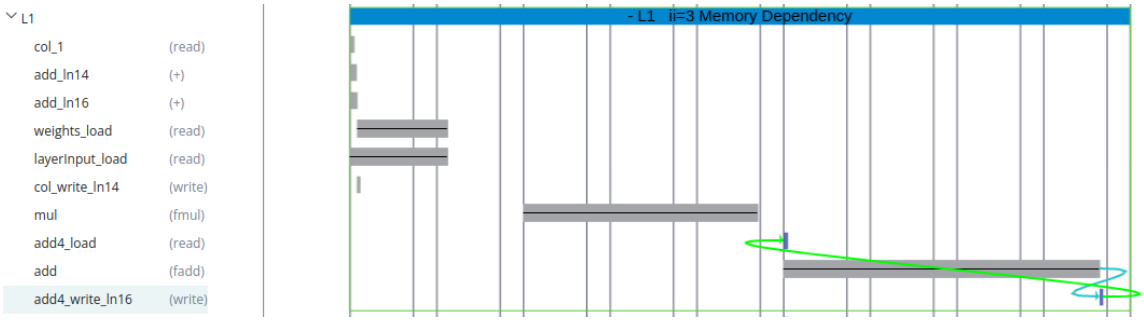

Figure 1: Schedule of the inner loop when pipelined **L1**.

An iteration of the whole loop taking 9 cycles, the pipelining should result in a division of the interval by 3. As the outer loop is not modified, the latency should be improved by the same factor and stay similar to the interval.

3.2.2 Outer Loop Pipelining

L0 can be pipelined in order to reduce the interval. This will unroll the inner loop as a pipelined loop must not contain any other loop. The function will now compute a new row of the output at each cycle, making the interval drop to OUT_SIZE[2].
Because **L1** is unrolled, each of its iterations is performed simultaneously. This means *weights* and *layerInput* should have multiple read operations happening at the same cycle. To do so,

[2]equal to 256 here

these arrays are partitioned along the accessed dimension, which is the first one for the input layer and the second one for the weights matrix.
Their interface is then set to be 1-port RAMs, allowing a single access per cycle. There is no need for more as only a single read is needed.
The resulting latency can be computed as the number of pipeline stages, to which the number of cycles of a single stage must be added. Fig. 2 gives an insight on how the inner loop has been flattened. As the operation is a multiplication followed by a sum, each input row is multiplied with the corresponding weight and is then added to the sum computed by the previous input row.

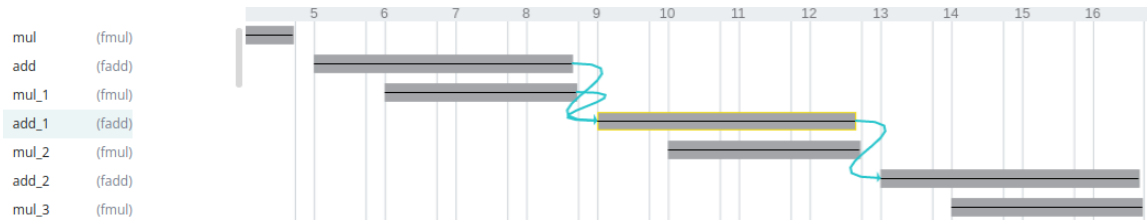

Figure 2: Partial schedule of the pipelined outer loop.

This addition takes 4 cycles and it must be applied for each input, so it is repeated IN_SIZE[3] times. This gives an approximate latency cycle count of $256 + 4 \times 256 = 1280$.

A quick note here: knowing the output is simply the sum of the effect of each input and the ordering of the sum has no effect, the latency could be reduced by summing the input contributions in a "tree-like" manner instead of a "chain-like" summation. The HLS tool is probably capable of such things but it goes outside of the scope of this course.

3.3 Area Oriented

For area optimization, the sequential code of Sec. 2.3 has been used to take advantage of the low resource consumption of FIFOs. To avoid duplicating computing elements, loop unrolling and pipelining have been disabled, together with the partitioning as parallel access is no longer needed.
The use of FIFOs removes the need for some flip flops (FF) where data does not need to be accessed multiple times, which makes it use less area.

3.4 Results

The 4 versions, differing only in the synthesis directives for the first 3 and with a slight code modification for the last one, were synthesized and the result is shown on Fig. 3.

[3] equal to 256 too

	Layer_0_Basic	Layer_1_InnerPipelining	Layer_2_OuterPipelining	Layer_3_FIFO
Component Information (5)				
Component Metrics (3)				
C SYNTHESIS (11)	✓	✓	✓	✓
Clock Target (ns)	10	10	10	10
Clock Uncertainty (ns)	2.7	2.7	2.7	2.7
Clock Achieved (ns)	7.016	7.016	7.016	7.016
Interval (cycles)	592386	200706	256	656130
Latency (cycles)	592385	200705	1291	656129
Latency (ns)	5924000	2007000.0000000002	12910	6561000
BRAM	0	0	0	1
DSP	5	5	1282	5
FF	635	642	167257	542
LUT	675	723	107551	660

Figure 3: Result of the different designs for a single layer.

The interval and latency for the inner and outer pipelining have already been explained and the last design has, without any surprise, worse performance metrics as they were traded for less components usage. The gain is mostly visible in the FF count.
Concerning the resources usage of the inner pipelining design, it only gains a few FFs and LUTs because only a small part of the code is pipelined. The outer pipelined design sees a major increase as the inner loop is unrolled, needing 256 times more resources[4].
Table 1 gives the improvement of each design with regards to the baseline and clearly show how much the outer pipelined design improved performance metrics and the FIFO design improved the area metrics.

Table 1: Performance Improvement Relative to Baseline

	Inner Pipelining	**Outer Pipelining**	**FIFOs**
Interval	33.88%	0.04%	110.76%
Latency	33.88%	0.22%	110.76%
DSP	$\times 1$	$\times 2.56$	$\times 1$
FF	$\times 1.01$	$\times 2.63$	$\times 0.85$
LUT	$\times 1.07$	$\times 1.59$	$\times 0.97$

3.5 About synthesis warnings

- The inner pipelining produced two warnings, telling the pipelined loop could reach an II of 1 and setting the synthesis directive to a target II of 3 resolved them.
- The FIFO implementation produced four warnings, one for each list that has been implemented as a FIFO. Those simply remind the designer to make sure the data is accessed in a sequential order, which has been checked here.

[4] $256 \times 642 = 164.352$ being quite close to the true FF count.

4 AE Optimization

The *flatten* and *unflatten* methods are similar in their structure to the other layers so the HLS directives used on those are the same.

4.1 Baseline

The baseline uses layers as defined in Sec. 2.2. Pipelining and loop unrolling is disabled and all function inlining is also prohibited.
Because the top module *AE* calls multiple functions, the synthesis tool gives performance statistics for each subfunction, as shown in Fig. 4.

Instance	Module	Latency (cycles) min	Latency (cycles) max	Latency (absolute) min	Latency (absolute) max	Interval min	Interval max
grp_flatten_fu_104	flatten	22009	22009	0.220 ms	0.220 ms	22009	22009
grp_inputLayer_fu_112	inputLayer	1807361	1807361	18.074 ms	18.074 ms	1807361	1807361
grp_intermediateLayer_fu_122	intermediateLayer	590849	590849	5.908 ms	5.908 ms	590849	590849
grp_endEncryptionLayer_fu_138	endEncryptionLayer	295425	295425	2.954 ms	2.954 ms	295425	295425
grp_startDecryptionLayer_fu_148	startDecryptionLayer	295937	295937	2.959 ms	2.959 ms	295937	295937
grp_outputLayer_fu_158	outputLayer	1808689	1808689	18.087 ms	18.087 ms	1808689	1808689
grp_unflatten_fu_168	unflatten	5545	5545	55.450 us	55.450 us	5545	5545

Figure 4: Performances statistics for baseline AE.

Because of the huge size of the input and output images (784 pixels), the layers at the boundaries of the model have the worst timing performance when no optimization is applied.

4.2 Performance Oriented

4.2.1 Outer Loop Pipelining

Based on the results for a single layer, it is known that pipelining the outer loop of each layer will give the best performances. By applying the same directives (pipeline + reshape + 1 port RAM) to *flatten* and *unflatten*, all loops are pipelined in the most aggressive manner, yielding the best performances for the design.

Instance	Module	Latency (cycles) min	Latency (cycles) max	Latency (absolute) min	Latency (absolute) max	Interval min	Interval max
grp_flatten_fu_17984	flatten	55	55	0.550 us	0.550 us	29	29
grp_inputLayer_fu_18828	inputLayer	3399	3399	33.990 us	33.990 us	257	257
grp_intermediateLayer_fu_21442	intermediateLayer	1287	1287	12.870 us	12.870 us	257	257
grp_endEncryptionLayer_fu_23243	endEncryptionLayer	1159	1159	11.590 us	11.590 us	129	129
grp_startDecryptionLayer_fu_24145	startDecryptionLayer	775	775	7.750 us	7.750 us	257	257
grp_outputLayer_fu_24791	outputLayer	1813	1813	18.130 us	18.130 us	785	785
grp_unflatten_fu_26349	unflatten	34	34	0.340 us	0.340 us	29	29

Figure 5: Performances statistics for outer loop pipelined AE.

Fig. 5 shows that each layer's interval is now equal to its output size, meaning the limiting operation of the whole model is the output layer with an interval of 785 cycles. However, Fig. 7 shows the interval is not the same as the slowest step of the design but equal to 12.403. This number is approximately found by summing the latency of each layer[5]: $55 + 3399 + 4 \times 1287 +$

[5]where the intermediateLayer latency is multiplied by 4 as the function is used 4 times

$1159 + 775 + 1813 + 34 = 12.383$.
This means the design waits for an input to have passed through all the layers before accepting a new one.

4.3 Dataflow

Applying the dataflow directive on the top module goes one step further as it will optimize how data is transmitted from a function to another and will pipeline the dataflow with one pipeline stage being defined by a function.
Taking a quick look at Fig. 6, it can immediately be seen that the intermediate layer is now implemented 4 times, once per call. Also, the dataflow directive gained one interval cycle fro each function.

Instance	Module	Latency (cycles) min	Latency (cycles) max	Latency (absolute) min	Latency (absolute) max	Interval min	Interval max
flatten_U0	flatten	55	55	0.550 us	0.550 us	28	28
inputLayer_U0	inputLayer	3399	3399	33.990 us	33.990 us	256	256
intermediateLayer_U0	intermediateLayer	1287	1287	12.870 us	12.870 us	256	256
intermediateLayer_1_U0	intermediateLayer_1	1287	1287	12.870 us	12.870 us	256	256
endEncryptionLayer_U0	endEncryptionLayer	1159	1159	11.590 us	11.590 us	128	128
startDecryptionLayer_U0	startDecryptionLayer	775	775	7.750 us	7.750 us	256	256
intermediateLayer_2_U0	intermediateLayer_2	1287	1287	12.870 us	12.870 us	256	256
intermediateLayer_3_U0	intermediateLayer_3	1287	1287	12.870 us	12.870 us	256	256
outputLayer_U0	outputLayer	1813	1813	18.130 us	18.130 us	784	784
unflatten_U0	unflatten	34	34	0.340 us	0.340 us	28	28

Figure 6: Performances statistics for dataflow AE.

The most important result being the interval of the whole AE function which dropped, as expected, to the one of the slowest subfunction, as showed again in Fig. 7.

4.4 Area Oriented

For area optimization, nothing new has been added compared to the single layer design. Loops stay as such and FIFOs are placed everywhere as all arrays are used in a sequential manner.

4.5 Results

	AE_0_Basic	AE_1_OuterPipelining	AE_2_Dataflow	AE_3_FIFO
Component Information (5)				
Component Metrics (3)				
C SYNTHESIS (11)	✓	✓	✓	✓
Clock Target (ns)	10	10	10	10
Clock Uncertainty (ns)	2.7	2.7	2.7	2.7
Clock Achieved (ns)	7.296	7.296	7.484	7.202
Interval (cycles)	6598382	12403	784	8056462
Latency (cycles)	6598381	12402	12392	8056461
Latency (ns)	65984000	124000	124000	80565000
BRAM	1316	2713	0	17
DSP	10	5200	12380	10
FF	2041	655788	1979828	2031
LUT	4402	459818	1302032	4173

Figure 7: Result of the different designs for a single layer.

The improvement with regards to the baseline design are summarized in Table 2.

Table 2: Performance Improvement Relative to Baseline

	Outer Pipelining	**Dataflow**	**FIFOs**
Interval	0.19%	0.01%	122.10%
Latency	0.19%	0.19%	122.10%
DSP	×520	×1238	×1
FF	×321.3	×970	×0.99
LUT	×104.4	×295.8	×0.95

Once again, the FIFO design has improved the area with regards to the baseline whereas the design with the best performances is the dataflow one.

4.6 About synthesis warnings

- The dataflow design presents 2 warnings regarding the clock target. Running the synthesis took close to 6 hours so it hasn't been run again with the patch, which consists of setting the target clock to 11ns as it now reaches 10.185ns (with the uncertainty).
- The dataflow design also warns the designer that deadlocks can occur, meaning a new input could interfere with the remainder of the previous data in the pipeline. A possible solution would be to add a cycle of interval to leave 1 cycle more than what is needed to avoid such issues.
- The FIFOs design warns about the 18 interfaces implemented as buffers but those are indeed called in a sequential order so there is no need to investigate the warning.